

4. Query Optimization

In a high-traffic flight system, the efficiency of SQL queries can be the difference between a smooth booking and a timed-out session. Query optimization begins with the strategic use of "indexes," which act like an index in a book, allowing the database to find data without scanning every single row. We specifically index "Search" fields such as departure airports and dates, as these are the most common entry points for users. Beyond indexing, we use "Query Execution Plans" to identify bottlenecks where the database might be performing "full table scans" instead of targeted lookups. We also practice "query refactoring," which involves rewriting complex joins into more efficient formats to reduce CPU and memory usage. For seat availability, we use "Covering Indexes" so the database can answer the query directly from the index without even touching the main table. Another technique is "materialized views" for historical data, which pre-calculates expensive totals like monthly revenue for administrative reports. We also limit the use of `SELECT *` to ensure we are only pulling the exact columns needed, reducing network overhead. By caching the results of frequent, non-changing queries—like the list of all active airports—we further reduce the load on the database. These optimizations collectively ensure that the system remains lightning-fast even as the number of flights in the database grows into the millions.

5. Transactions & Concurrency

Transactions are the mechanism that ensures a flight booking is an "all or nothing" operation. If a user pays for a ticket but the system fails to update the seat status, the transaction is "rolled back," and the money is refunded to maintain consistency. We use the **BEGIN TRANSACTION**, **COMMIT**, and **ROLLBACK** commands to wrap these critical operations together. Concurrency control is managed through "isolation levels," which determine how visible one user's changes are to another user at the same time. To prevent double-booking, we employ "pessimistic locking" on specific seat rows during the final checkout phase.

This ensures that while one user is typing their credit card details, that specific seat is invisible to all other searchers. We also monitor for "deadlocks," a rare situation where two transactions are waiting for each other to release a lock, by implementing a timeout and retry logic. "Optimistic concurrency control" is used for less critical updates, like changing a passenger's frequent flyer number, where conflicts are unlikely. By managing these simultaneous interactions carefully, we ensure that the database remains stable under the pressure of thousands of concurrent users. This part of the system is what guarantees that the airline never sells the same seat twice. It turns a chaotic environment of global clicks into an orderly, sequential series of confirmed actions.

6. Security & Access Control

Security is implemented through a "defense-in-depth" strategy, starting with Role-Based Access Control (RBAC). In this model, a 'Customer' can only see their own bookings, while 'Staff' can modify flight times, and 'Admins' have full system control. This prevents unauthorized users from accessing sensitive backend data or altering pricing structures. We use "Parameterized Queries" to protect against SQL Injection, ensuring that user input is never executed as code. All passwords are "hashed" using strong algorithms like BCrypt, meaning even if the database is stolen, the actual

passwords remain unreadable. For financial data, we use "Tokenization," where the database stores a reference token instead of the actual credit card number. Data "at rest" is encrypted on the server disks, and data "in transit" is protected by industry-standard TLS encryption. We also maintain "Audit Logs," which record every change made to the database, including who made the change and when. This allows us to track down any suspicious activity or accidental errors made by staff members. Frequent security patches and vulnerability scans are part of the maintenance plan to stay ahead of emerging threats. By treating security as a foundational element rather than an afterthought, we build a system that passengers can trust with their private information.